Explain Pigeonhole Principle

Today 10:32 PM

Pigeonhole Principle (PHP) 🕊️📦

The **Pigeonhole Principle** is a very simple but powerful idea used in **mathematics, competitive programming, data structures, and algorithms**.

1. Basic Idea

Imagine:

- You have **more pigeons than pigeonholes**.
- Every pigeon must sit in a hole.

If there are **6 pigeons** and only **5 holes**, then at least one hole must contain **2 or more pigeons**.

Mathematical statement

> If **n + 1 objects** are placed into **n boxes**, then at least one box contains **at least 2 objects**.

For example:

```
6 pigeons → 5 holes

P1 → H1
P2 → H2
P3 → H3
P4 → H4
P5 → H5
P6 → ?

No empty new hole exists.

Therefore:

Some hole must contain ≥ 2 pigeons.
```

2. Generalized Pigeonhole Principle

The basic principle only tells us that some box has at least 2 objects.

But what if we have many more objects?

Suppose:

- `N` objects
- `K` boxes

Then at least one box contains at least:

$$\left\lceil \frac{N}{K} \right\rceil$$

objects.

Where `ceil` means **round up**.

Example

Suppose:

```
10 pigeons
3 holes
```

$$\left\lceil \frac{10}{3} \right\rceil = \lceil 3.33 \rceil = 4$$

Therefore:

> At least one hole must contain **at least 4 pigeons**.

Because the most evenly distributed arrangement is:

```
Hole 1 → 3
Hole 2 → 3
```

```
Hole 3 → 4
```

You cannot distribute 10 pigeons among 3 holes such that every hole has at most 3.

---

3. The Main Formula

If:

- `N` = number of objects
- `K` = number of boxes/categories

Then:

$$\boxed{\text{Maximum minimum guaranteed objects in one box} = \left\lceil \frac{N}{K} \right\rceil}$$

In programming:

```
C++
int answer = (N + K - 1) / K;
```

This is integer ceiling division.

Example:

```
C++
N = 10
K = 3

(10 + 3 - 1) / 3
= 12 / 3
= 4
```

4. Reverse Form

Sometimes the question asks:

> How many objects are required to guarantee that some box contains at least `X` objects?

Suppose:

- `K` boxes
- We want at least `X` objects in one box.

To avoid having `X` objects in any box, each box can contain at most:

$$X - 1$$

So total objects without guarantee:

$$K(X - 1)$$

Therefore, to guarantee one box has `X` objects:

$$\boxed{K(X - 1) + 1}$$

Example

There are `5` boxes.

How many objects are needed to guarantee that one box has at least `4` objects?

Each box can have at most:

```
3 objects
```

Maximum without reaching 4:

```
5 × 3 = 15
```

Add one more:

```
16
```

Therefore:

$$\boxed{16}$$

objects guarantee that at least one box has **4** objects.

---

5. The Most Important Skill: Identify Pigeons and Holes

In problems, the biggest challenge is identifying:

🕊️ Pigeons = Objects

Things that are being distributed.

🕳️ Holes = Categories

Groups where objects can belong.

For example:

> There are 13 people. Prove that at least two were born in the same month.

Pigeons

```
13 people
```

Holes

```
12 months
```

Since:

$$13 > 12$$

At least two people must share a birth month.

---

6. Important Properties of Pigeonhole Principle

Property 1: More objects than categories ⇒ repetition

If:

$$N > K$$

then at least one category contains more than one object.

Example:

```
101 students
100 possible marks

At least two students have the same mark.
```

Property 2: General distribution guarantee

If **N** objects are distributed into **K** categories:

$$\boxed{\text{At least one category has } \left\lceil \frac{N}{K} \right\rceil}$$

objects.

Property 3: Another useful guarantee

There is also at least one box containing **at most**:

$$\boxed{\left\lfloor \frac{N}{K} \right\rfloor}$$

objects.

Example

```
10 objects
3 boxes
```

At least one box has:

```
ceil(10/3) = 4 or more
```

And at least one box has:

```
floor(10/3) = 3 or fewer
```

Property 4: Worst-case thinking

Pigeonhole problems are mostly about:

> **What is the worst possible distribution that avoids the required condition?**

Then add **1**.

For example:

> How many people are required to guarantee 5 people were born in the same month?

Try avoiding 5 people in every month.

Maximum per month:

```
4 people
```

So:

```
12 × 4 = 48
```

With 48 people, possible:

```
Jan → 4
Feb → 4
...
Dec → 4
```

Nobody has 5.

Add one:

$$\boxed{49}$$

Therefore, 49 people guarantee 5 people in the same birth month.

---

7. Pigeonhole Principle in Mathematics

Application 1: Same remainder

Suppose we have:

```
11 integers
```

When divided by **10**, possible remainders are:

```
0,1,2,3,4,5,6,7,8,9
```

There are only:

```
10 possible remainders
```

But we have:

```
11 numbers
```

Therefore, at least two numbers must have the same remainder.

Pigeons

```
11 integers
```

Holes

```
10 possible remainders
```

Therefore:

$$\boxed{\text{At least two integers have the same remainder modulo } 10}$$

---

8. Application: Two Numbers With Divisible Difference

Suppose we have `n + 1` integers.

Consider their remainders when divided by `n`.

Possible remainders:

```
0, 1, 2, ..., n-1
```

There are only `n` possibilities.

But there are `n + 1` integers.

Therefore, two numbers have the same remainder.

Suppose:

$$a \mod n = b \mod n$$

Then:

$$a - b$$

is divisible by `n`.

So:

$$\boxed{\text{Among n+1 integers, two have a difference divisible by n}}$$

This idea appears frequently in **number theory and DSA problems**.

---

9. Application in Strings

Suppose you have binary strings of length `n`.

Each position has:

```
0 or 1
```

So total possible strings:

$$2^n$$

If you have:

$$2^n + 1$$

binary strings of length `n`, then at least two must be identical.

Example

Binary strings of length `3`:

```
000
001
010
011
100
101
110
111
```

Only:

$$2^3 = 8$$

possible strings.

If you have 9 strings:

$$\boxed{\text{At least two must be identical}}$$

10. Pigeonhole Principle in Arrays

Consider:

```
C++
vector<int> nums;
```

Suppose:

```
Array size = n + 1
Each element is between 1 and n
```

There are:

```
n possible values
```

but:

```
n + 1 elements
```

Therefore:

$$\boxed{\text{At least one duplicate must exist}}$$

This is the mathematical reason behind problems like:

Find the Duplicate Number.

For example:

```
nums = [1, 3, 4, 2, 2]
```

Possible values:

```
1, 2, 3, 4
```

4 holes.

But:

```
5 elements
```

5 pigeons.

Therefore, duplicate is guaranteed.

11. Application in Hashing 🔥

A hash table maps many possible keys into a limited number of buckets.

Example:

```
1,000,000 possible keys
10,000 buckets
```

By the Pigeonhole Principle:

> Different keys must eventually map to the same bucket.

This is called a:

$$\boxed{\text{Hash Collision}}$$

```
Keys                    Hash Buckets

Key A ──────────────► Bucket 0
Key B ──────────────► Bucket 1
Key C ──────────────► Bucket 1  ← Collision
Key D ──────────────► Bucket 2
```

Since the number of possible keys is usually much larger than the number of buckets, collisions are **mathematically unavoidable**.

12. Application in Hash Maps

Suppose:

```
1000 elements
100 buckets
```

Then at least one bucket contains at least:

$$\left\lceil \frac{1000}{100} \right\rceil = 10$$

elements.

So PHP can be used to reason about the **worst-case distribution**.

```
1000 elements
      ↓
100 buckets

At least one bucket
contains ≥ 10 elements
```

Of course, this doesn't mean every bucket has 10 elements; it only guarantees that **at least one** does.

---

13. Application in Graph Theory

Suppose a graph has `n` vertices.

Every vertex has a degree between:

$$0 \text{ and } n-1$$

At first glance, there are `n` possible degree values.

But something interesting happens:

You cannot simultaneously have:

```
degree 0
```

and:

```
degree n-1
```

Because:

- Degree `0` means a vertex connects to nobody.
- Degree `n-1` means a vertex connects to everybody.

These cannot both exist.

Therefore, there are effectively only:

$$n-1$$

possible degree categories.

But we have:

$$n$$

vertices.

By the Pigeonhole Principle:

$$\boxed{\text{At least two vertices have the same degree}}$$

This is a classic graph theory application.

---

14. Application in Prefix Sum + HashMap 🚀

This is one of the most important DSA applications.

Suppose:

```
arr = [a1, a2, a3, ..., an]
```

Define prefix sum:

$$prefix[i] = a_1 + a_2 + \cdots + a_i$$

Suppose two prefix sums have the same remainder modulo **k**.

$$prefix[i] \mod k = prefix[j] \mod k$$

Then:

$$prefix[j] - prefix[i]$$

is divisible by **k**.

But:

$$prefix[j] - prefix[i] = \text{sum of subarray } (i+1 \ldots j)$$

Therefore:

$$\boxed{\text{There exists a subarray whose sum is divisible by k}}$$

Example

```
arr = [2, 3, 1, 4]
k = 5
```

Prefix sums:

```
Prefix Sum     Remainder mod 5

0                0
2                2
5                0
6                1
10               0
```

Notice:

```
0 remainder repeats
```

For prefix:

```
0 → 5
```

Difference:

```
5 - 0 = 5
```

Divisible by **5**.

This idea is used in problems such as:

- Subarray Sum Divisible by K
- Continuous Subarray Sum
- Count Subarrays With Sum Divisible by K

---

15. Application in Cycle Detection

Consider a linked list where every node points to at most one next node.

If you keep moving:

```
node → next → next → next...
```

and the number of steps becomes greater than the number of unique nodes, then by the Pigeonhole Principle:

> You must visit some node again.

Once a node repeats:

$$\boxed{\text{A cycle exists}}$$

This is the underlying mathematical intuition behind cycle detection.

For example:

```
A → B → C → D
        ↑       ↓
        ← ← ← ←
```

If you keep visiting nodes:

```
A, B, C, D, C, D, C, D...
```

There are finite nodes but infinitely continuing visits.

Eventually, nodes must repeat.

Algorithms such as **Floyd's Tortoise and Hare** exploit this finite-state/repetition idea.

---

16. Application in Finite State Problems

Suppose an algorithm can only have `N` possible states.

If the algorithm runs for more than `N` transitions without terminating, then:

$$\boxed{\text{Some state must repeat}}$$

Why?

```
N possible states
N + 1 visits
```

Therefore, at least one state is visited twice.

This is useful in:

- Cycle detection
- Dynamic programming state analysis
- Graph traversal
- Automata
- Simulation problems

---

17. Application in Birthday Problem

There are approximately:

```
365 possible birthdays
```

If you have:

```
366 people
```

then at least two must have the same birthday.

```
366 people > 365 possible birthdays
```

Therefore:

$$\boxed{\text{At least two share a birthday}}$$

If leap years are considered, use 367 people for an unconditional guarantee.

---

18. Application in Competitive Programming

When you see constraints like:

```
Array has n + 1 elements
Values are between 1 and n
```

Immediately think:

> **Pigeonhole Principle → Duplicate must exist.**

Another common pattern:

```
n + 1 numbers
```

and:

```
only n possible remainders
```

Immediately think:

> Two numbers have the same remainder.

Then:

```
same remainder
        ↓
difference divisible by modulus
```

Another pattern:

```
More prefixes than possible states/remainders
```

Then:

> Some prefix state must repeat.

This usually means:

```
Repeated prefix state
        ↓
A special subarray exists
```

19. The Core DSA Pattern

A very useful mental model is:

```
            MANY OBJECTS
                 │
                 ▼
        Classify each object
                 │
                 ▼
        LIMITED CATEGORIES
                 │
                 ▼
        Objects > Categories?
            /         \
          YES          NO
           │
           ▼
    SOME CATEGORY REPEATS
           │
           ▼
    What does repetition mean?
           │
           ▼
    Duplicate / Collision /
    Cycle / Valid Subarray /
    Same Remainder / Same State
```

This is the main connection between the Pigeonhole Principle and algorithms.

---

20. Important Interview/DSA Examples

Here are some patterns you should remember:

Pattern	Pigeons	Holes	Result
Duplicate in array	**n+1** elements	**n** values	Duplicate exists
Same remainder	Numbers	**k** remainders	Two same remainder
Prefix sum modulo	Prefixes	**k** remainders	Divisible subarray
Hashing	Keys	Hash buckets	Collision
Graph degrees	Vertices	Possible degrees	Same degree
Cycle detection	Visits	Nodes/states	State repeats
Strings	Strings	Possible combinations	Duplicate string
Birthday	People	Days	Same birthday

---

21. A Powerful Problem-Solving Method

Whenever you see a problem, ask these **4 questions**:

Step 1: What are the pigeons?

What objects am I distributing?

```
Numbers?
People?
Array elements?
Prefix sums?
Graph vertices?
States?
Strings?
```

Step 2: What are the holes?

What are the limited categories?

```
Values?
Remainders?
Buckets?
Degrees?
States?
Possible strings?
```

Step 3: Compare them

```
Objects > Categories?
```

If yes, repetition is guaranteed.

Step 4: Interpret the repetition

This is the most important part.

For example:

```
Two equal array values
        ↓
Duplicate

Two equal remainders
        ↓
Difference divisible by k

Two equal prefix remainders
        ↓
Subarray divisible by k

Two equal graph states
        ↓
Cycle/repetition

Two hash keys in same bucket
        ↓
Collision
```

Final Cheat Sheet 🧠

Basic PHP

$$\boxed{n+1 \text{ objects in } n \text{ boxes} \Rightarrow \text{one box has at least } 2 \text{ objects}}$$

Generalized PHP

$$\boxed{N \text{ objects in } K \text{ boxes} \Rightarrow \text{one box has at least } \left\lceil \frac{N}{K} \right\rceil}$$

Guarantee at least X objects in one box

$$\boxed{K(X-1)+1}$$

Competitive programming mindset

```
FINITE NUMBER OF POSSIBILITIES
            +
MORE OBJECTS/STATES THAN POSSIBILITIES
            ↓
      REPETITION MUST OCCUR
            ↓
Find what that repetition implies
```

The **Pigeonhole Principle itself is simple**. The real skill is recognizing what the **pigeons**, **holes**, and **meaning of the repeated category** are. In DSA, that repeated category often reveals a **duplicate, collision, cycle, equal remainder, or special subarray**.

Today 11:34 PM

Combinatorics in Math + DSA 🚀

Combinatorics is the branch of mathematics that deals with **counting**.

In DSA, whenever you see questions like:

- How many ways?
- How many arrangements?
- How many subsets?
- How many strings?
- How many paths?
- Count all possible valid configurations?

👉 You are probably entering the world of **combinatorics**.

The most important concepts are:

```
COMBINATORICS
├── Counting Principle
├── Factorial
├── Permutation → Order matters
├── Combination → Order does NOT matter
├── Pigeonhole Principle
├── Inclusion-Exclusion
└── Counting with DP / Recursion
```

Let's build this from zero.

---

1. What is Combinatorics?

Suppose you have:

```
A, B, C
```

You want to count how many different ways you can select or arrange them.

For example:

Select 2

```
AB
AC
BC
```

Answer = **3**.

Arrange 2

```
AB
BA
AC
CA
BC
CB
```

Answer = **6**.

Why are the answers different?

Because in the first case:

```
AB = BA
```

We only care about **which elements were selected**.

But in the second:

```
AB ≠ BA
```

We care about the **order**.

This gives us the most important rule:

🔥 Permutation vs Combination

```
Does ORDER matter?
        │
    ┌───┴────┐
   YES       NO
    │        │
Permutation Combination
```

2. Fundamental Counting Principle

Before permutation and combination, understand this.

Suppose:

- You have `n` choices for the first operation.
- For every choice, you have `m` choices for the second operation.

Then total ways:

$$\boxed{n \times m}$$

Example

You have:

```
3 shirts
2 pants
```

How many outfits?

For every shirt, you can choose any of the 2 pants.

```
        Pants
          │
Shirt A ──┼── 2 choices
Shirt B ──┼── 2 choices
Shirt C ──┼── 2 choices
```

Therefore:

$$3 \times 2 = 6$$

This is called the **Multiplication Principle**.

Addition Principle

If choices are mutually exclusive:

```
Choose either A or B
```

and:

```
A → 3 ways
B → 5 ways
```

Then:

$$\boxed{3 + 5 = 8}$$

DSA connection

In recursion:

```
C++
solve()
{
    // Option 1
    ways += solve(...);

    // Option 2
    ways += solve(...);
}
```

You are often using the **addition principle**.

If recursion branches like:

```
                State
               /     \
              /       \
        Choice A   Choice B
```

Total ways:

$$ways(A) + ways(B)$$

But if a process has multiple independent choices:

```
Position 1 → 3 choices
Position 2 → 4 choices
Position 3 → 2 choices
```

Then:

$$3 \times 4 \times 2$$

---

3. Factorial `n!`

Factorial is heavily used in permutations.

$$\boxed{n! = n(n-1)(n-2)\dots 2 \cdot 1}$$

Example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Meaning:

> The number of ways to arrange `n` distinct objects is `n!`.

For:

```
A B C
```

Arrangements:

```
ABC
ACB
BAC
BCA
CAB
CBA
```

Total:

$$3! = 6$$

Important properties

1.

$$\boxed{0! = 1}$$

This is important in combinatorics and DP.

Why?

There is exactly **one way to arrange nothing**.

2.

$$n! = n \times (n-1)!$$

This is the recursive definition.

```C++
long long factorial(int n) {
    if (n <= 1) return 1;

    return n * factorial(n - 1);
```

```
}
```

3. Factorial grows extremely fast

```
5!  = 120
10! = 3,628,800
20! ≈ 2.43 × 10^18
```

This is important in DSA because:

```
O(n!)
```

usually becomes impossible quickly.

Example:

```
n = 10 → 3.6 million permutations
n = 15 → 1.3 trillion permutations 😵
```

4. Permutation 🔀

A **permutation** means arranging objects.

> **Order matters.**

Suppose:

```
A, B, C
```

Choose and arrange 2.

```
AB
BA
AC
CA
BC
CB
```

Notice:

```
AB ≠ BA
```

Therefore, answer = **6**.

Formula for Permutation

If we have:

- **n** total objects
- Choose **r** objects
- Order matters

Then:

$$\boxed{P(n,r) = \frac{n!}{(n-r)!}}$$

Also written as:

$$^nP_r$$

Why does this formula work?

Suppose:

```
n = 5
r = 3
```

We have:

```
A B C D E
```

For the first position:

```
5 choices
```

For the second:

```
4 choices
```

For the third:

```
3 choices
```

Therefore:

$$5 \times 4 \times 3 = 60$$

Using the formula:

$$\frac{5!}{(5-3)!} = \frac{5!}{2!} = 5 \times 4 \times 3 = 60$$

Visual intuition

```
5 objects: A B C D E

Position 1 → 5 choices
              |
Position 2 → 4 choices
              |
Position 3 → 3 choices

Total = 5 × 4 × 3
```

5. Permutation Properties

Property 1

$$P(n, n) = n!$$

Because you are arranging all elements.

Property 2

$$P(n, 1) = n$$

Choose one object.

Property 3

$$P(n, r) = n \times (n-1) \times \ldots (n-r+1)$$

This is often easier in code.

```
C++
long long permutation(int n, int r) {
    long long ans = 1;

    for (int i = 0; i < r; i++) {
        ans *= (n - i);
    }

    return ans;
}
```

6. Combination 🎯

A **combination** means selecting objects.

> **Order does NOT matter.**

Suppose:

```
A, B, C
```

Choose 2:

```
AB
AC
BC
```

We do **not** count:

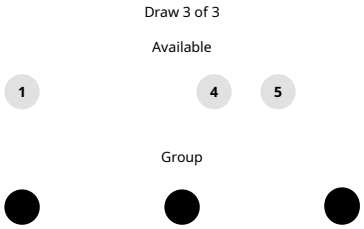
```
BA
CA
CB
```

because:

```
AB = BA
```

in combinations.

Formula



$\binom{6}{3} = \frac{6!}{3!(6-3)!} = 20$
After all 3 draws finish, their numerical order shows one combination; changing draw order does not create a new one.
Options 6 Draws 3
Give feedback

$$\boxed{C(n,r) = \frac{n!}{r!(n-r)!}}$$

Also written as:

$$^nC_r$$

---

7. Why Do We Divide by `r!`?

Let's understand this deeply.

Suppose:

```
n = 3
r = 2
```

Permutation gives:

```
AB
BA

AC
CA

BC
CB
```

Total:

$$6$$

But in combinations:

```
AB = BA
```

Each group is counted:

$$2! = 2$$

times.

Therefore:

$$\frac{6}{2!} = 3$$

So:

$$\boxed{C(n,r) = \frac{P(n,r)}{r!}}$$

And:

$$C(n,r) = \frac{n!}{(n-r)!r!}$$

8. Permutation vs Combination

Question	Permutation	Combination
Does order matter?	Yes	No
AB and BA	Different	Same
Main purpose	Arrange	Select
Formula	$n!/(n-r)!$	$n!/[r!(n-r)!]$
Example	Password	Team selection
DSA example	Permutations of array	Generate subsets

Easy trick 🧠

```
PERMUTATION
P → Position matters

COMBINATION
C → Choose
```

Not a strict mathematical rule, but a useful memory trick.

9. Most Important Combination Properties

Property 1: Symmetry

$$\boxed{C(n,r) = C(n,n-r)}$$

Example:

$$C(5,2) = C(5,3) = 10$$

Why?

Choosing:

```
2 elements to include
```

is the same as choosing:

```
3 elements to exclude
```

Example:

```
A B C D E
```

Choose:

```
A B
```

You are automatically excluding:

```
C D E
```

So:

$$\boxed{\text{Choose r} = \text{Leave n-r}}$$

Property 2

$$C(n, 0) = 1$$

There is exactly one way to choose nothing.

Property 3

$$C(n, n) = 1$$

There is exactly one way to choose everything.

Property 4: Pascal's Identity 🔥

$$\boxed{C(n, r) = C(n-1, r-1) + C(n-1, r)}$$

This is extremely important in **DP**.

Intuition

Suppose we have `n` elements and focus on one particular element `X`.

Two possibilities:

Case 1: Take X

Then we need:

```
r - 1
```

elements from the remaining `n - 1`.

Ways:

$$C(n-1, r-1)$$

Case 2: Don't take X

Then choose all `r` elements from remaining `n-1`.

Ways:

$$C(n-1, r)$$

Therefore:

$$\boxed{C(n, r) = C(n-1, r-1) + C(n-1, r)}$$

---

10. This Is Directly Connected to Recursion

Consider generating combinations.

At every element:

```
              Element X
              /       \
             /         \
        Take X       Don't Take X
```

Therefore:

```
C++
int solve(int n, int r) {
    if (r == 0 || r == n)
        return 1;

    return solve(n - 1, r - 1)
```

```
        + solve(n - 1, r);
}
```

This is literally Pascal's Identity.

11. Memoization Version

```
C++
class Solution {
public:

    vector<vector<int>> dp;

    int solve(int n, int r) {

        // Invalid case
        if (r < 0 || r > n)
            return 0;

        // Base cases
        if (r == 0 || r == n)
            return 1;

        if (dp[n][r] != -1)
            return dp[n][r];

        return dp[n][r] =
            solve(n - 1, r - 1) +
            solve(n - 1, r);
    }

    int nCr(int n, int r) {
        dp.assign(n + 1, vector<int>(r + 1, -1));

        return solve(n, r);
    }
};
```

Time:

$$O(nr)$$

12. Tabulation Version

```
C++
int nCr(int n, int r) {

    vector<vector<long long>> dp(
        n + 1,
        vector<long long>(r + 1, 0)
    );

    for (int i = 0; i <= n; i++) {

        dp[i][0] = 1;

        for (int j = 1; j <= min(i, r); j++) {

            if (j == i)
                dp[i][j] = 1;
            else
                dp[i][j] =
                    dp[i - 1][j - 1] +
                    dp[i - 1][j];
        }
    }

    return dp[n][r];
}
```

13. Pascal's Triangle ▲

The values of **`nCr`** form Pascal's Triangle.

```
                1
            1       1
        1       2       1
    1       3       3       1
1       4       6       4       1
  1     5    10    10    5    1
```

Every number is:

```
top-left + top-right
```

Example:

$$10 = 4 + 6$$

This triangle appears in:

- DP
- Binomial coefficients
- Probability
- Counting
- Mathematical identities

---

14. Connection Between Combination and Subsets

Suppose an array has:

```
n elements
```

How many subsets?

For every element:

```
Take
or
Don't Take
```

So:

$$2 \times 2 \times 2 \dots$$

n times:

$$\boxed{2^n}$$

Another way:

Number of subsets with:

```
0 elements → C(n,0)
1 element  → C(n,1)
2 elements → C(n,2)
...
n elements → C(n,n)
```

Total:

$$C(n,0) + C(n,1) + \cdots + C(n,n)$$

And:

$$\boxed{\sum_{r=0}^{n} C(n,r) = 2^n}$$

This is a very important identity.

---

15. Application: Generate All Subsets

```
C++
void solve(
    int index,
    vector<int>& nums,
    vector<int>& current,
    vector<vector<int>>& ans
) {
    if (index == nums.size()) {
        ans.push_back(current);
        return;
    }

    // Don't take
    solve(index + 1, nums, current, ans);

    // Take
    current.push_back(nums[index]);

    solve(index + 1, nums, current, ans);

    current.pop_back();
}
```

The recursion tree has:

$$\boxed{2^n}$$

possible subsets.

---

16. Application: Generate All Permutations

For:

```
[A, B, C]
```

First position:

```
3 choices
```

Second:

```
2 choices
```

Third:

```
1 choice
```

Therefore:

$$3! = 6$$

Backtracking:

```
C++
void solve(
    vector<int>& nums,
    int index,
    vector<vector<int>>& ans
) {
    if (index == nums.size()) {
        ans.push_back(nums);
        return;
    }

    for (int i = index; i < nums.size(); i++) {

        swap(nums[index], nums[i]);

        solve(nums, index + 1, ans);

        swap(nums[index], nums[i]);
    }
}
```

Time complexity is approximately:

$$O(n \cdot n!)$$

because there are `n!` permutations and copying/outputting each takes up to `O(n)`.

---

17. Application: Grid Paths 🗺️

Suppose you want to move from:

```
(0,0)
```

to:

```
(m,n)
```

and can only move:

```
Right → R
Down  → D
```

You must make:

```
m Down moves
n Right moves
```

Total moves:

$$m + n$$

Now we just need to decide:

> Which `m` positions contain Down?

Therefore:

$$\boxed{C(m+n, m)}$$

Or:

$$C(m+n, n)$$

Example:

```
2 Right
2 Down
```

Total moves = **4**.

Ways:

$$C(4, 2) = 6$$

Possible paths:

```
RRDD
RDRD
RDDR
DRRD
DRDR
DDRR
```

This is a beautiful example of converting a **path problem into a combination problem**.

---

18. Application: Strings

Suppose a binary string has length `n`.

Each position:

```
0 or 1
```

Therefore:

$$2^n$$

total binary strings.

If exactly `r` positions must contain `1`:

Choose those `r` positions:

$$\boxed{C(n, r)}$$

Example:

Length:

```
n = 5
```

Exactly:

```
r = 2 ones
```

Answer:

$$C(5, 2) = 10$$

Because you are choosing which 2 positions contain **1**.

---

19. Application: Frequency / Duplicate Permutations

Suppose:

```
A A B
```

Naively:

$$3! = 6$$

But swapping the two **A**s doesn't create a new arrangement.

So:

$$\boxed{\frac{3!}{2!} = 3}$$

Unique permutations:

```
AAB
ABA
BAA
```

General Formula

If:

- Total elements = **n**
- Frequencies = $f_1, f_2, f_3, \ldots$

Then:

$$\boxed{\frac{n!}{f_1! f_2! f_3! \ldots}}$$

Example:

```
A A B B C
```

Total:

$$5!$$

Repeated:

```
A → 2 times
B → 2 times
C → 1 time
```

Answer:

$$\frac{5!}{2!2!1!} = 30$$

This is useful in:

- Unique permutations
- Anagram problems
- String counting

---

20. Combination in Probability

Suppose:

```
10 people
```

Choose:

```
3 people
```

for a team.

Order doesn't matter.

$$C(10, 3)$$

But suppose:

```
President
Vice President
Secretary
```

Order/role matters.

$$P(10,3)$$

This distinction is extremely useful in probability.

---

21. Modular Combinatorics in Competitive Programming 🔥

This is one of the most important DSA applications.

Problems often ask:

> Return the answer modulo $10^9 + 7$.

We cannot directly calculate factorial for huge `n`.

We use:

$$C(n,r) = \frac{n!}{r!(n-r)!}$$

But division under modulo requires modular inverse.

Precompute:

```
fact[i]    = i!
invFact[i] = inverse(i!)
```

Then:

$$\boxed{C(n,r) = fact[n] \times invFact[r] \times invFact[n-r] \pmod{MOD}}$$

C++ Template

```
C++
class Combinatorics {
public:
    static const long long MOD = 1e9 + 7;

    vector<long long> fact, invFact;

    long long power(long long a, long long b) {
        long long ans = 1;

        while (b) {
            if (b & 1)
                ans = ans * a % MOD;

            a = a * a % MOD;
            b >>= 1;
        }

        return ans;
    }

    Combinatorics(int n) {

        fact.resize(n + 1);
        invFact.resize(n + 1);

        fact[0] = 1;

        for (int i = 1; i <= n; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }

        invFact[n] = power(fact[n], MOD - 2);

        for (int i = n; i > 0; i--) {
            invFact[i - 1] = invFact[i] * i % MOD;
        }
    }

    long long nCr(int n, int r) {

        if (r < 0 || r > n)
            return 0;

        return fact[n]
             * invFact[r] % MOD
             * invFact[n - r] % MOD;
    }
};
```

Complexity

Precomputation:

$$O(N)$$

Each `nCr` query:

$$\boxed{O(1)}$$

This is extremely useful when there are many combination queries.

---

22. Important Combinatorial Identities for DSA

1. Symmetry

$$\boxed{C(n,r) = C(n,n-r)}$$

Always use:

```
C++
r = min(r, n - r);
```

when calculating directly.

---

2. Pascal Identity

$$\boxed{C(n,r) = C(n-1,r-1) + C(n-1,r)}$$

Used in:

- Recursion
- Memoization
- DP
- Pascal triangle

---

3. Sum of all combinations

$$\boxed{\sum_{r=0}^{n} C(n,r) = 2^n}$$

Used for:

- Subsets
- Bitmasking
- Take/not-take recursion

---

4. Weighted identity

$$\boxed{\sum_{r=0}^{n} r \cdot C(n,r) = n \cdot 2^{n-1}}$$

This can appear in advanced counting problems.

Intuition:

Across all subsets of `n` elements, each element appears in exactly half:

$$2^{n-1}$$

subsets.

There are `n` elements, so total element appearances:

$$n \cdot 2^{n-1}$$

---

23. Combination With Repetition

Sometimes you can select an item multiple times.

Example:

You have 3 types of candies:

```
A, B, C
```

Choose 5 candies.

You can choose:

```
A A A B C
```

or:

```
B B B B B
```

The formula is:

$$\boxed{C(n+r-1, r)}$$

where:

- `n` = number of types
- `r` = number of objects selected

This is called **Stars and Bars**.

Example:

```
3 candy types
5 candies
```

Answer:

$$C(3+5-1, 5) = C(7, 5) = 21$$

---

24. Stars and Bars 🌟

Suppose:

```
x + y + z = 5
```

where:

$$x, y, z \geq 0$$

How many solutions?

Imagine 5 stars:

```
* * * * *
```

We need to divide them into 3 groups.

For that, we need:

```
2 bars
```

Example:

```
**|***|
```

Means:

```
x = 2
y = 3
z = 0
```

Total symbols:

```
5 stars + 2 bars = 7
```

Choose positions of the 2 bars:

$$\boxed{C(7,2)}$$

In general:

$$\boxed{x_1 + x_2 + \cdots + x_k = n}$$

with:

$$x_i \geq 0$$

Number of solutions:

$$\boxed{C(n+k-1, k-1)}$$

Very useful in advanced CP problems.

---

25. Combinatorics and Bitmasking

An array with `n` elements has:

$$2^n$$

subsets.

A bitmask represents:

```
0 → don't take
1 → take
```

Example:

```
A B C
```

```
000 → {}
001 → {C}
010 → {B}
011 → {B,C}
100 → {A}
101 → {A,C}
110 → {A,B}
111 → {A,B,C}
```

This is combinatorics implemented directly using binary.

```
C++
for (int mask = 0; mask < (1 << n); mask++) {

    for (int i = 0; i < n; i++) {

        if (mask & (1 << i)) {
            // nums[i] belongs to this subset
        }
    }
}
```

26. Where Combinatorics Appears in DSA

```
COMBINATORICS
│
├── Backtracking
│   ├── Subsets → 2^n
│   ├── Permutations → n!
│   └── Combinations → nCr
│
├── Dynamic Programming
│   ├── Count paths
│   ├── Count ways
│   ├── Pascal identity
│   └── Take / Not Take
│
├── Strings
│   ├── Anagrams
│   ├── Unique permutations
│   └── String arrangements
│
├── Graphs
│   ├── Count paths
│   └── Count possible edges
│
├── Bit Manipulation
│   ├── Subsets
│   └── Bitmask DP
│
└── Number Theory
    ├── nCr modulo M
    ├── Modular inverse
```

```
└── Factorials
```

27. How to Recognize a Combinatorics Problem 🧠

Whenever you see:

Pattern 1

How many ways?

Think:

```
Addition or Multiplication principle?
```

Pattern 2

Select `r` from `n`

Think:

$$C(n, r)$$

Pattern 3

Arrange `r` from `n`

Think:

$$P(n, r)$$

Pattern 4

Every element can be selected or not selected

Think:

$$2^n$$

Pattern 5

Move only Right and Down

Think:

```
Total moves = Right + Down

Choose positions of:
Right moves
or
Down moves
```

Then use:

$$nCr$$

Pattern 6

Repeated characters/items

Think:

$$\frac{n!}{f_1!f_2!\dots}$$

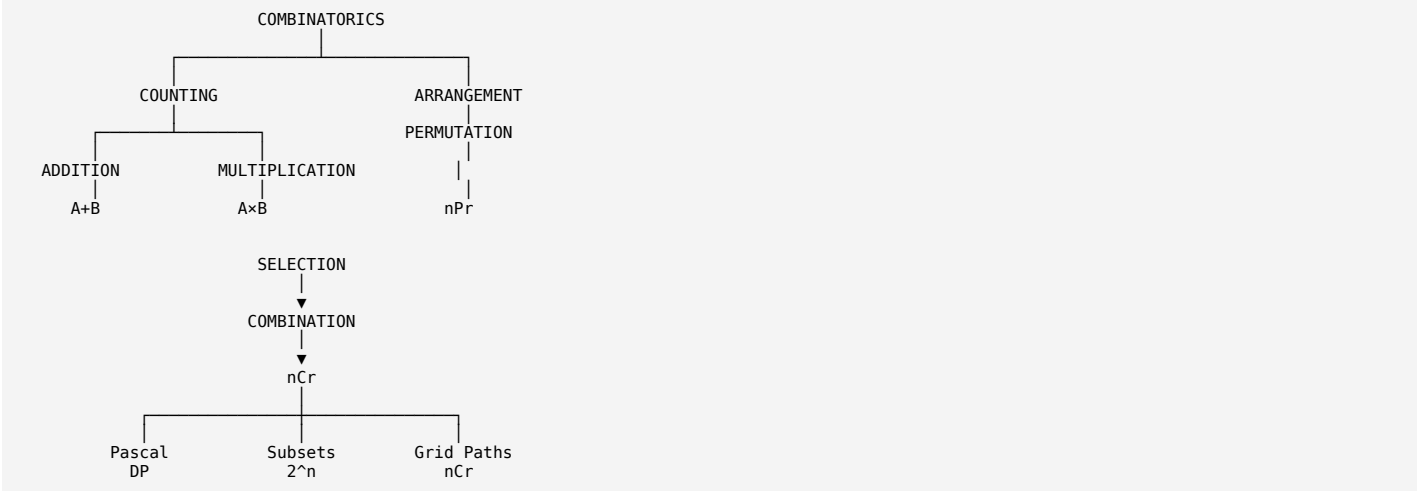
Pattern 7

Count distributions

Think:

```
Stars and Bars
```

Final Mental Map 🔥



The one-line difference you should never forget

> **Permutation = Select + Arrange → order matters.**
> **Combination = Select only → order does not matter.**

And in DSA, the biggest practical connection is:

```
"How many ways?"
      ↓
Can I model choices?
      ↓
Order matters? ── Yes → Permutation
      │
      No
      ↓
Combination / Subsets / DP counting
```